

DAY 12 SQL

2) Common Table Expressions (CTE)

Theory:

Declared with WITH keyword. Exists only till the query runs. Does not store data; instead, re-runs each time. Good for breaking complex queries, improving readability, and supporting recursion.

Code:

```
with SalesEMPbyCountry as (  
    select salesRepEmployeeNumber, country, sum(creditLimit) as total  
    from customers  
    group by country, salesRepEmployeeNumber  
)  
empdetails as (  
    select * from SalesEMPbyCountry  
    join employees on salesRepEmployeeNumber = employeeNumber  
)  
select * from empdetails;
```

3) Temporary Tables

Theory: Lives till session ends. Stored physically, so faster for repeated access. Only visible to the user who created it. Can be dropped manually. Takes priority over permanent tables if name clashes.

Code:

```
create temporary table CountryWiseAnanlysis as  
select count(customerNumber) as totalCustomer, country, sum(creditLimit) as total  
from customers  
group by country, salesRepEmployeeNumber;  
select * from CountryWiseAnanlysis;
```

4) Transactions

Theory:

A transaction is a logical unit of work. It ensures data integrity using ACID properties.

COMMIT: Saves all changes permanently.

ROLLBACK: Undo all changes (like Ctrl+Z).

SAVEPOINT: Partially rollback to a defined point.

Code:

```
create temporary table test(id int, name varchar(255));
```

```
start TRANSACTION;
```

```
insert into test values (1,'Kirat'), (2,'Abhi');
```

```
insert into test values (3,'Akash');
```

```
select * from test;
```

```
rollback; -- Undo changes
```

```
commit; -- Save changes
```

```
savepoint saveOne;
```

```
insert into test values (4,'Rahul');
```

```
rollback to saveOne;
```

6) Indexes

Theory: Clustered Index: Data arranged physically (usually on primary key). Non-Clustered Index: Creates a separate structure for fast lookup. Improves query performance but increases storage overhead.

Code:

```
-- Non-clustered index
```

```
create index idx_marks on Students_1(marks);
```

```
-- Clustered index
```

```
create index idx_creditLimit on customers(creditLimit);
```

7) Stored Procedures

Theory: Stored block of SQL statements with a name. Improves reusability and reduces redundant code. Can take parameters (IN, OUT).

Code:

```
delimiter //
```

```

create procedure CheckCreditLimit(in id int)
begin
    select customerName, creditLimit
    from customers
    where customerNumber = id;
end //
delimiter ;
call CheckCreditLimit(114);
drop procedure CheckCreditLimit;

```

8) Triggers

Theory: A trigger is an automatic execution of SQL code in response to INSERT, UPDATE, or DELETE. Useful for logging, validation, and enforcing rules.

Works row-wise (FOR EACH ROW).

Code:

```

create table orders_1(order_id int, Order_date Date, Sdate Date default null);
create table order_log(id int, status varchar(255), logtime datetime default now());
delimiter //
create trigger InsertOrder
after insert on orders_1
for each row
begin
    insert into order_log(id, status)
    values (new.order_id, 'In-progress');
end //
delimiter ;

```